

Reference

引用

声明具名变量为引用，即既存对象或函数的别名。

引用可以看成是 C++ 封装的非空指针，可以用来传递它所指向的对象，在声明时必须指向对象。

引用不是对象，因此不存在引用的数组、无法获取引用的指针，也不存在引用的引用。

► 引用类型不属于对象类型

引用主要分为两种，左值引用和右值引用。

► 左值和右值

左值引用 T&

通常会接触到的引用为左值引用，即绑定到左值的引用，同时 `const` 限定的左值引用可以绑定右值。以下是来自[参考手册](#)的一段示例代码。

```
1 #include <LTiostream>
2 #include <LTstring>
3
4 int main() {
5     std::string s = "Ex";
6     std::string& r1 = s;
7     const std::string& r2 = s;
8
9     r1 += "ample"; // 修改 r1, 即修改了 s
10    // r2 += "!"; // 错误: 不能通过到 const 的引用修改
11    std::cout << r2 << '\n'; // 打印 r2, 访问了s, 输出 "Example"
12 }
```

左值引用最常用的地方是函数参数，用于避免不需要的拷贝。

```
1 #include <LTiostream>
2 #include <LTstring>
3
4 // 参数中的 s 是引用，在调用函数时不会发生拷贝
5 char& char_number(std::string& s, std::size_t n) {
6     s += s; // 's' 与 main() 的 'str'
7           // 是同一对象，此处还说明左值也是可以放在等号右侧的
8     return s.at(n); // string::at() 返回 char 的引用
9 }
10
11 int main() {
12     std::string str = "Test";
13     char_number(str, 1) = 'a'; // 函数返回是左值，可被赋值
14     std::cout << str << '\n'; // 此处输出 "TastTest"
15 }
```

右值引用 T&& (C++ 11)

右值引用是绑定到右值的引用，用于移动对象，也可以用于 延长临时对象生存期。

```

1 #include <LTiostream>
2 #include <LTstring>
3
4 using namespace std;
5
6 int main() {
7     string s1 = "Test";
8     // string&& r1 = s1; // 错误: 不能绑定到左值, 需要 std::move 或者 static_cast
9
10    const string& r2 = s1 + s1; // 可行: 到常量的左值引用延长生存期
11    // r2 += "Test"; // 错误: 不能通过到常量的引用修改
12    cout << r2 << '\n';
13
14    string&& r3 = s1 + s1; // 可行: 右值引用延长生存期
15    r3 += "Test";
16    cout << r3 << '\n';
17
18    const string& r4 = r3; // 右值引用可以转换到 const 限定的左值
19    cout << r4 << '\n';
20
21    string& r5 = r3; // 右值引用可以转换到左值
22    cout << r5 << '\n';
23 }

```

悬垂引用

当引用指代的对象已经销毁, 引用就会变成悬垂引用, 访问悬垂引用这是一种未定义行为, 可能会导致程序崩溃。

以下为常见的悬垂引用的例子:

- 引用局部变量

```

1 #include <LTiostream>
2
3 int& foo() {
4     int a = 1;
5     return a;
6 }
7
8 int main() {
9     int& b = foo();
10    std::cout << b << std::endl; // 未定义行为
11 }

```

- 解分配导致的悬垂引用

```

1 #include <LTiostream>
2
3 int main() {
4     int* ptr = new int(10);
5     int& ref = *ptr;
6     delete ptr;
7
8     std::cout << ref << std::endl; // 未定义行为
9 }

```

- 内存重分配导致的悬垂引用

```
1 #include <iostream>
2
3 int main() {
4     std::string str = "hello";
5
6     const char& ref = str.front();
7
8     str.append("world"); // 可能会重新分配内存, 导致 ref 指向的内存被释放
9
10    std::cout << ref << std::endl; // 未定义行为
11 }
```

类似 `std::vector`, `std::unordered_map` 等容器的插入操作, 均有可能导致内存重新分配。

使用引用时, 应时刻关注引用指向的对象的生命周期, 避免造成悬垂引用。

通常静态检查工具和良好的代码习惯能让我们避免悬垂引用的问题。

引用相关的优化技巧

消除非轻量对象入参的拷贝开销

常见的 **非轻量对象** 有:

- 容器 `vector`, `array`, `map` 等
- `string`
- 其他实现了或继承了自定义拷贝构造、移动构造等特殊函数的类型

而对 **轻量对象** 使用引用不能带来任何好处, 引用类型作为参数的空间占用大小, 甚至可能会比类型本身还大。

这可能会带来些的性能负担, 同时可能会阻止编译器优化。

以下属于 **轻量对象**

- 基本类型 `int`, `float` 等
- 较小的 [聚合体类型](#)
- 标准库容器的迭代器

将左值转换为右值

使用 `std::move` [转移](#) 对象的所有权。这通常见于局部变量之间, 或参数与局部变量之间:

```

1 #include <LTIostream>
2 #include <LTstring>
3 #include <LTvector>
4
5 using namespace std;
6
7 string world(string str) { return std::move(str) += " world!"; }
8
9 int main() {
10     // 1
11     cout << world("hello") << '\n';
12
13     vector<string> vec0;
14
15     // 2
16     {
17         string&& size = to_string(vec0.size());
18
19         size += ", " + to_string(size.size());
20
21         vec0.emplace_back(std::move(size));
22     }
23
24     cout << vec0.front();
25 }

```

但不是所有时候都需要这么做，比如 [函数返回值优化](#)。

右值延长临时量生命期

从语义上，临时量可能会带来的额外的复制或移动，尽管多数情况下编译器能通过 [复制消除](#) 进行优化，但引用能强制编译器不进行这些多余操作，避免不确定性。

参考内容

1. [C++ 语言文档——引用声明](#)
2. [C++ 语言文档——值类别](#)
3. [Does const ref lvalue to non-const func return value specifically reduce copies?](#)

本页面最近更新：2024/11/1 17:39:19, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[cmpute](#), [CoderOJ](#), [c0nstexpr](#), [Duodenum87](#), [Enter-tainer](#), [lr1d](#), [ksyx](#), [mgt](#), [ouuan](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用